



MinorFlow

A gem5 Pipeline Visualizer for
Teaching Computer Architecture

Manuel Nieto · Francisco Cortez Casini · María Delfina Vélez Ibarra ·

Gonzalo Tomás Vodanovic

FaMAF, Universidad Nacional de Córdoba · UTN, Facultad Regional Villa María
Universidad Nacional de Córdoba, Argentina · 22nd September 2026

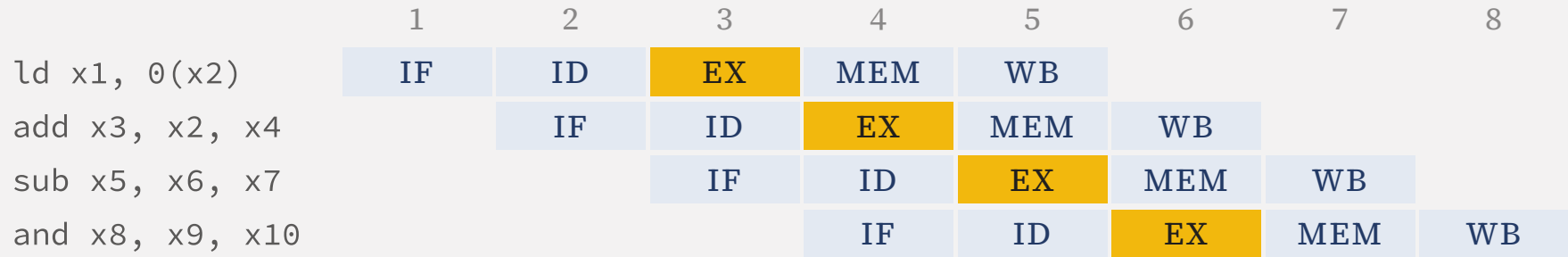
The gap between the textbook and the simulator

- A great number of undergraduate courses teach the simplified five-stage pipeline of Hennessy and Patterson's *Computer Architecture: A Quantitative Approach*. However, the processors that students meet afterwards do not behave like it.
- gem5 exposes realistic microarchitecture, but the standard exercise ends at the aggregate counters in `stats.txt`. Those counters report the outcome, but the per-cycle events that produced it stay in the log.

	1	2	3	4	5	6	7	8
ld x1, 0(x2)	IF	ID	EX	MEM	WB			
add x3, x2, x4		IF	ID	EX	MEM	WB		
sub x5, x6, x7			IF	ID	EX	MEM	WB	
and x8, x9, x10				IF	ID	EX	MEM	WB

Rows are instructions,
columns are cycles.
MinorFlow keeps this
convention.

THE FIVE-STAGE PIPELINE · IN DETAIL



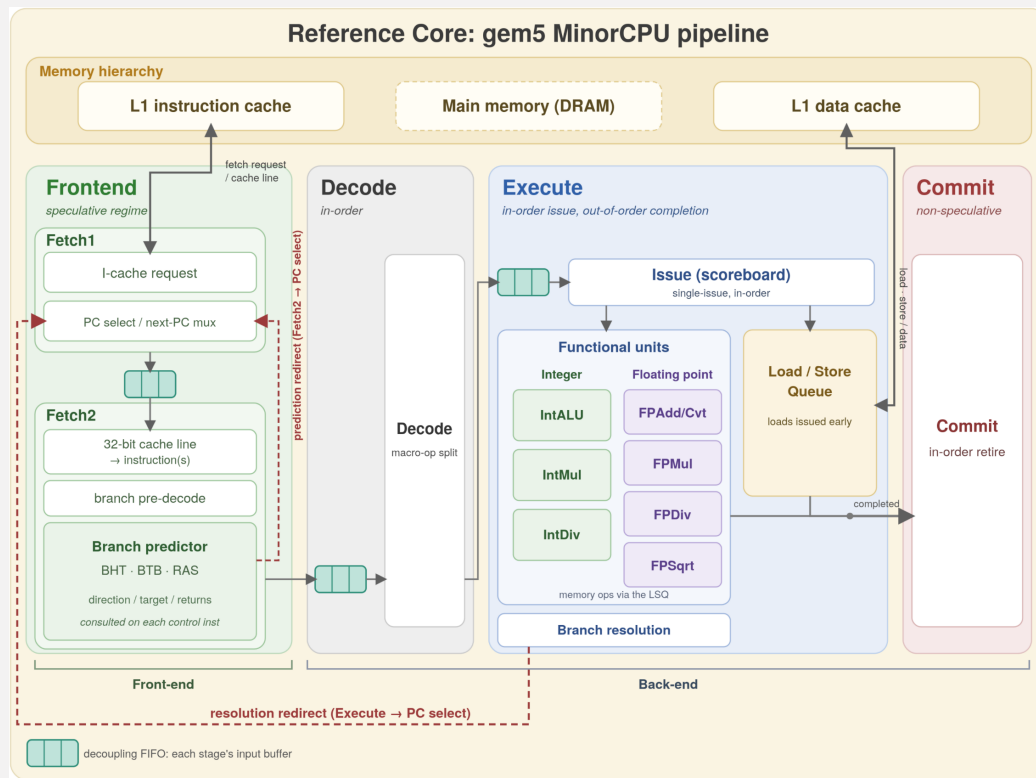
The Reference Core

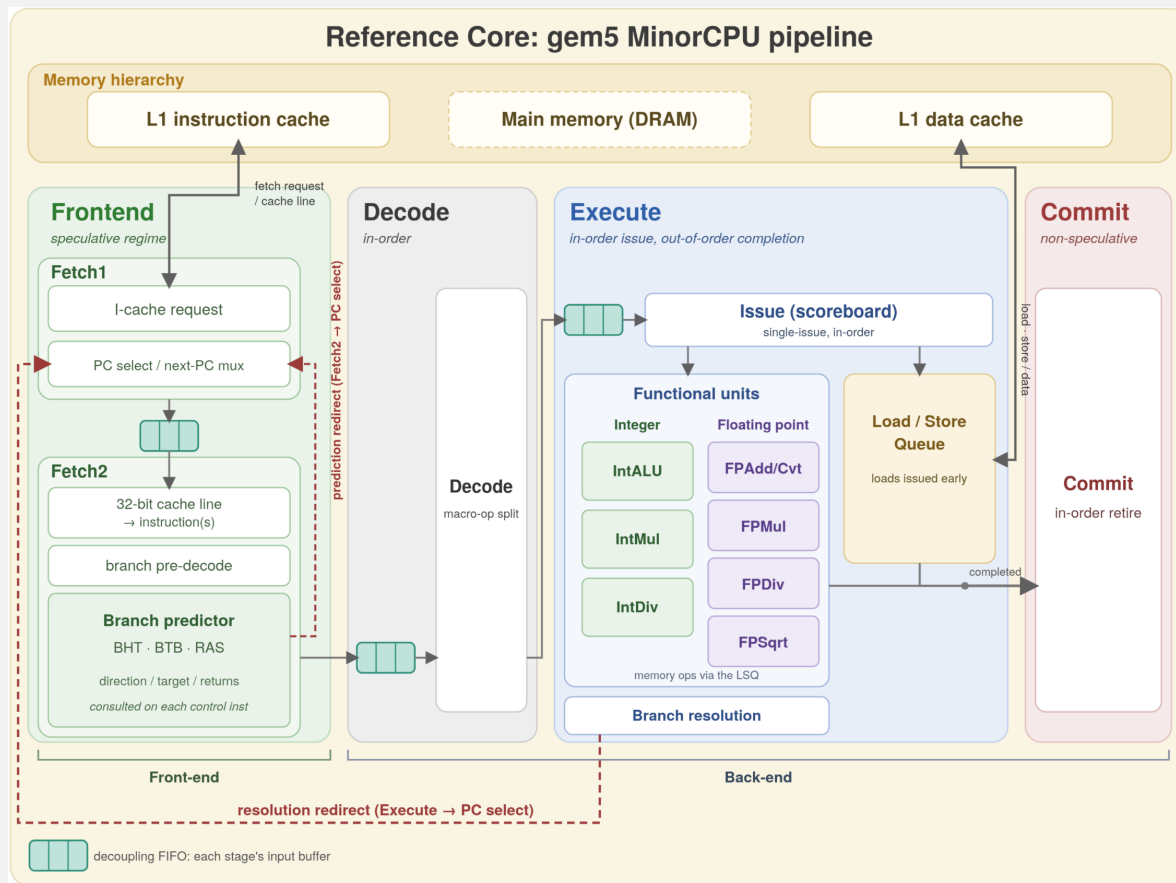
MinorCPU in a four-stage in-order configuration: single-issue, 64-bit RISC-V, 100 MHz base clock.

Parameterised after CVA6, the open six-stage in-order RISC-V application core.

Two ways the PC is redirected: a prediction at Fetch2, and a resolution at Execute.

A decoupling FIFO at every stage input, scoreboard issue, and a load-store queue that issues loads early.





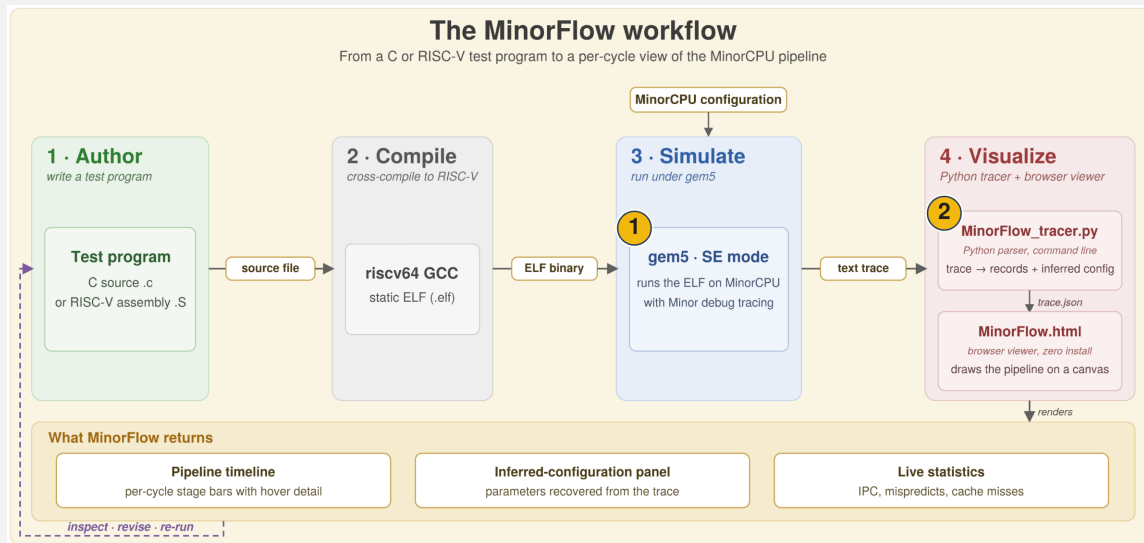


Existing tools and what they leave out

TOOL	WHAT IT SHOWS	WHAT IT LEAVES OUT
minorview.py (gem5)	Single-cycle state of the MinorCPU	Python 2 GTK with no multi-cycle view
o3-pipeview.py (gem5)	Multi-cycle text diagram	O3CPU only and static text
Konata	Interactive multi-cycle diagram	O3PipeView and Kanata only
Ripes, WebRISC-V, BRISC-V	Didactic five-stage simulators	Own pipeline with no external traces

No existing tool reads MinorTrace, runs in a browser with no installation, and draws a multi-cycle diagram.

The MinorFlow workflow

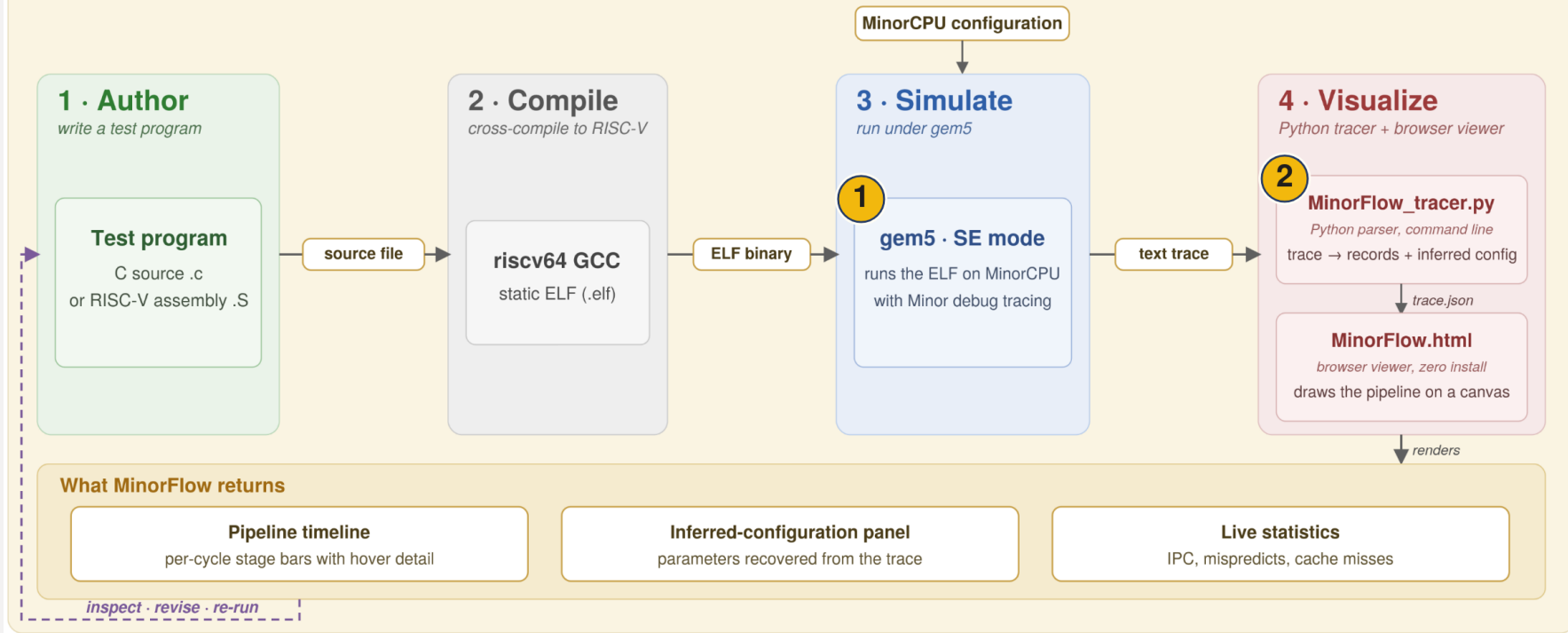


```
1 ~/gem5 $ build/RISCV/gem5.opt --debug-flags=Minor,MinorTrace,MinorTiming,CacheAll,ExecAll,
Fetch,Decode,IEW,Commit,LSQ,Scoreboard,Writeback
--debug-file=trace.txt minorcpu_config.py code.elf

2 ~/gem5 $ python3 MinorFlow_tracer.py m5out/trace.txt -o trace.json
```

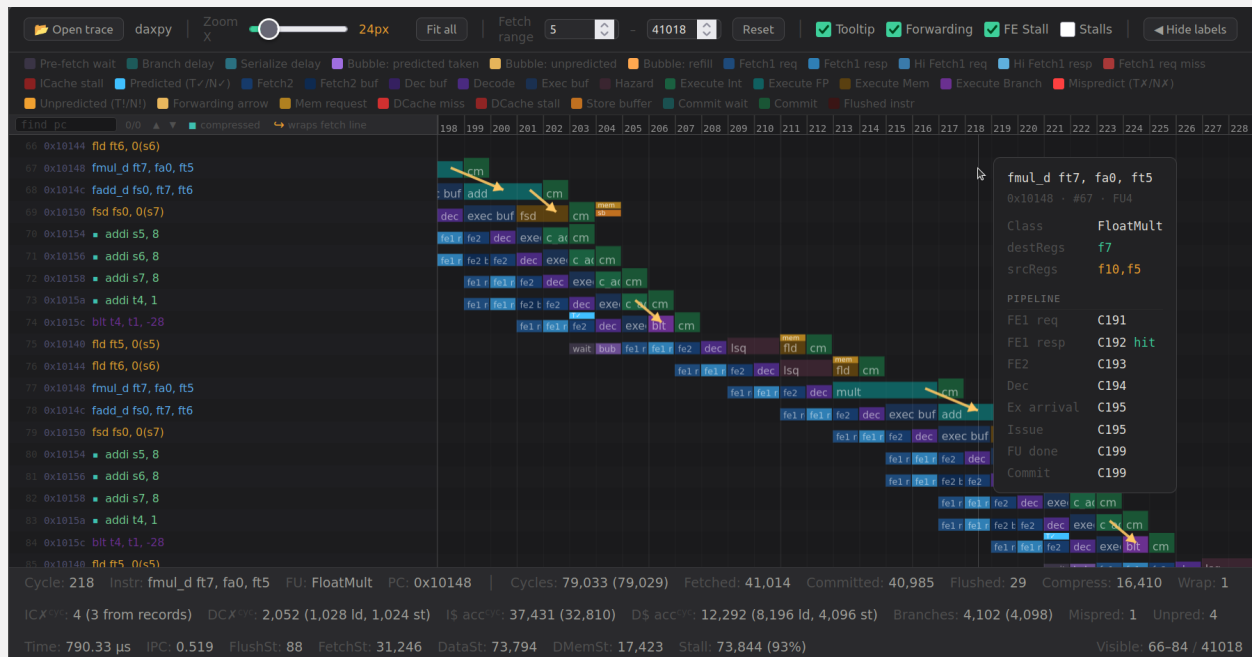
The MinorFlow workflow

From a C or RISC-V test program to a per-cycle view of the MinorCPU pipeline

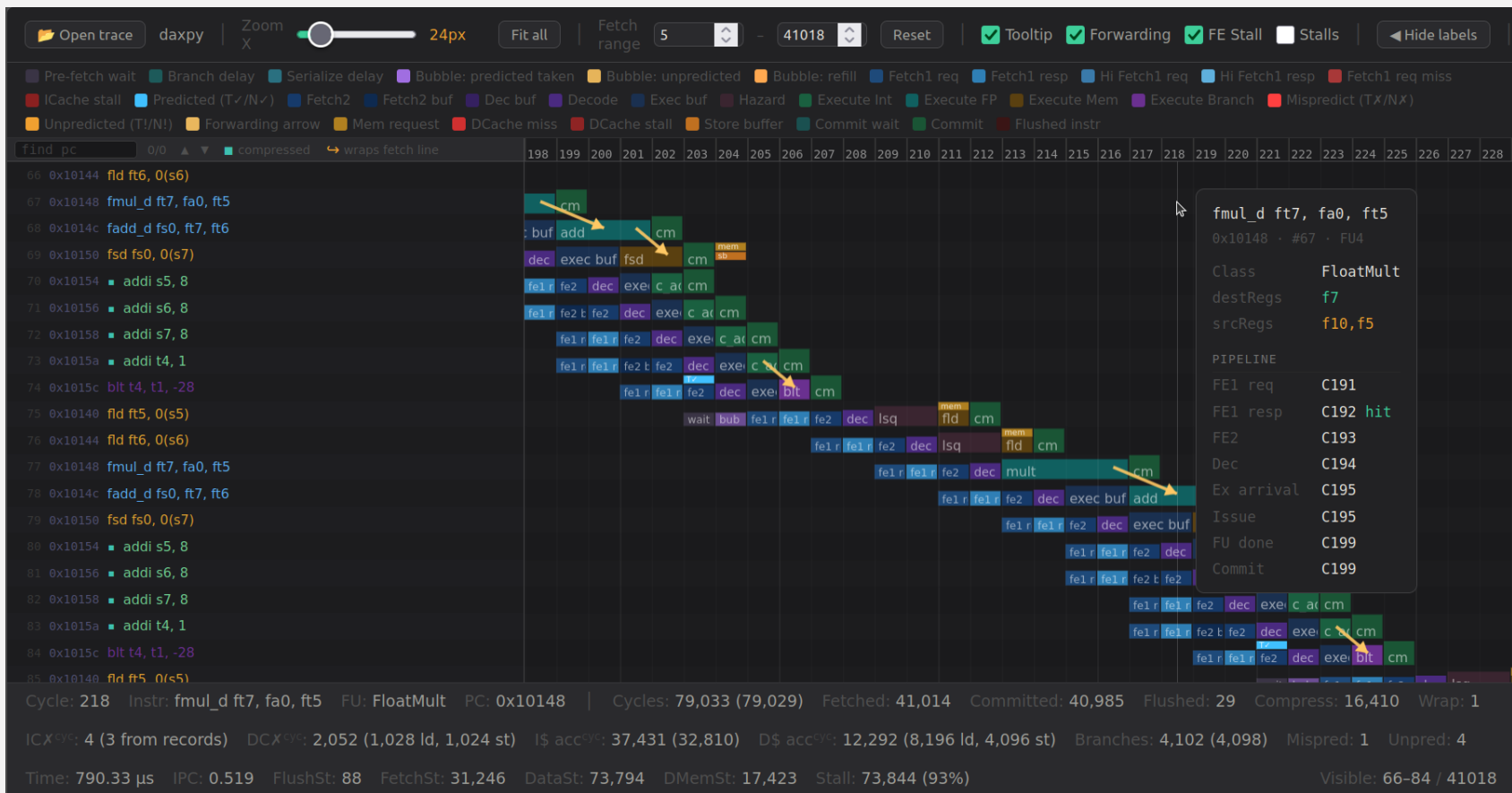


The timeline: one row per instruction

Rows in fetch order, columns in cycles. Stage bars, cache events and branch outcomes on the same row. The per-instruction inspector opens on hover.

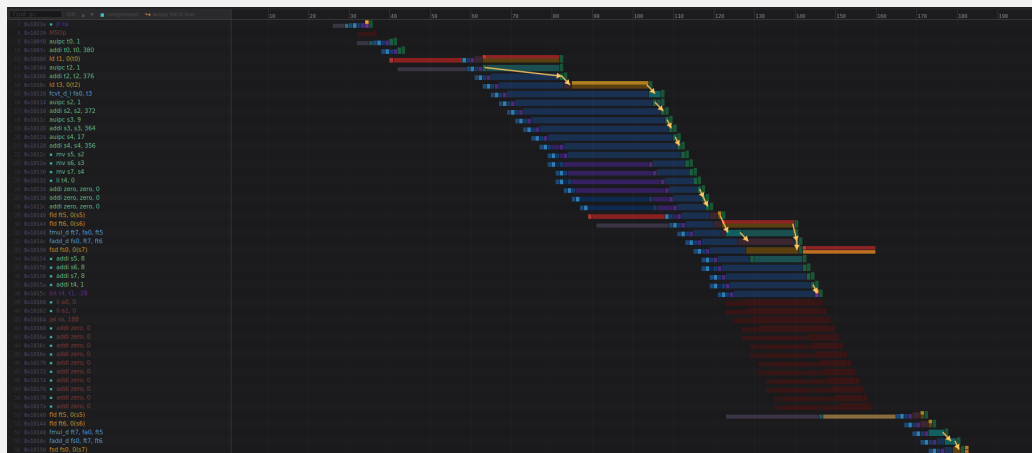


THE MINORFLOW VIEWER · IN DETAIL

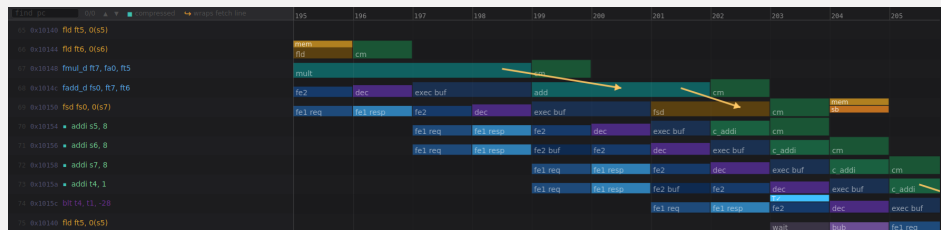


Reading the same trace at two scales

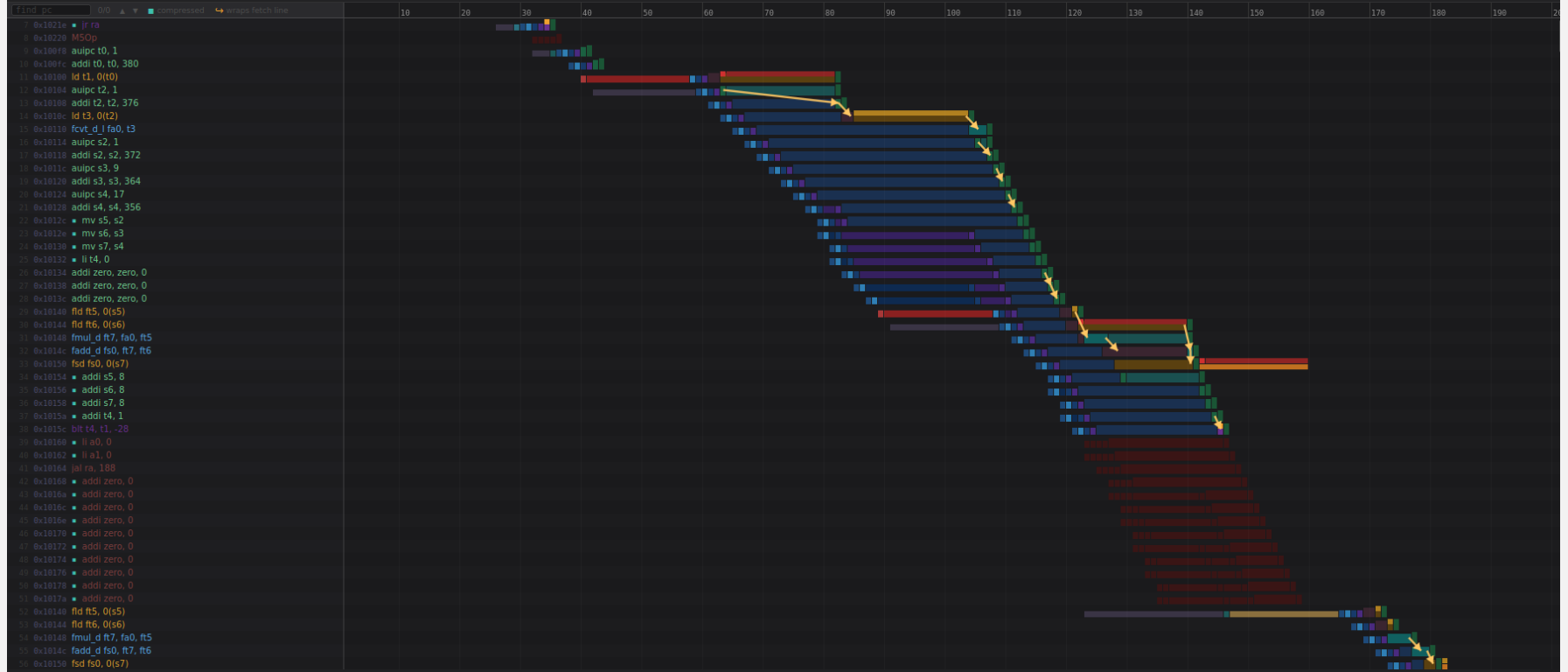
LOW ZOOM · a whole region of the program, and the shape it makes



HIGH ZOOM · one loop iteration, with the forwarding overlay



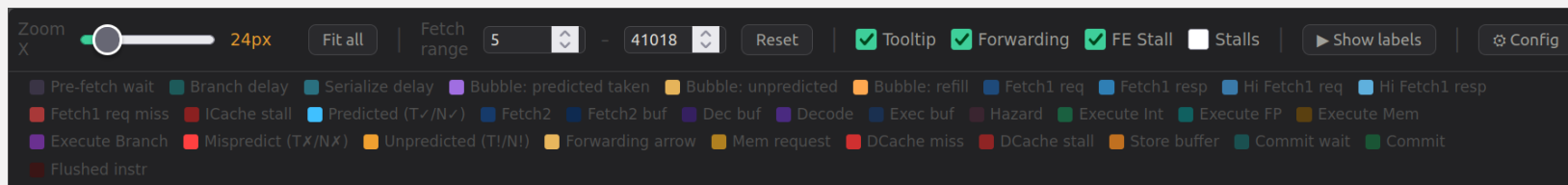
LOW ZOOM · IN DETAIL



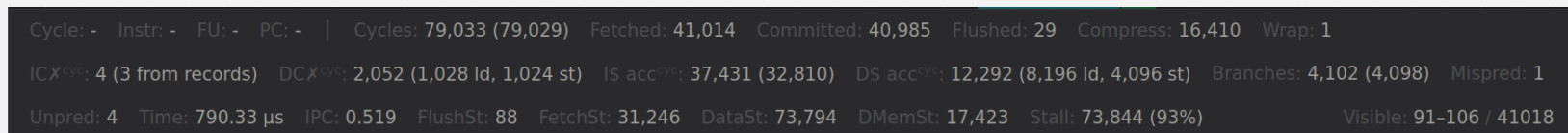


Focus controls and running statistics

TOOLBAR · fetch-range window, overlay toggles, config panel, stage legend



STATUS BAR · recomputed for the fetch-range window



Cycles, IPC, cache accesses and misses, branch totals with mispredicted and unpredicted outcomes, and the share of cycles lost to stalls.

Validation on daxpy

METRIC	HAND ESTIMATE	GEM5 STATS.TXT	MINORFLOW
Committed instructions	$\approx 40,960$	40,985	40,985
IPC	≈ 0.50	0.519	0.519
D-cache load misses	$\approx 1,024$	1,028	1,028

daxpy, $z = a \cdot x + y$, for $N = 4,096$ double-precision elements. The two measured columns agree exactly. The hand estimate models the inner loop alone, so the setup code accounts for the difference.

The same three numbers, from both sources

MINORFLOW · STATUS BAR, DERIVED FROM THE TRACE

```
Cycle: - Instr: - FU: - PC: - | Cycles: 79,033 (79,029) Fetched: 41,014 Committed: 40,985 Flushed: 29 Compress: 16,410 Wrap: 1
ICXOPS: 4 (3 from records) DCXOPS: 2,052 (1,028 ld, 1,024 st) I$ accOPS: 37,431 (32,810) D$ accOPS: 12,292 (8,196 ld, 4,096 st) Branches: 4,102 (4,098) Mispred: 1
Unpred: 4 Time: 790.33 μs IPC: 0.519 FlushSt: 88 FetchSt: 31,246 DataSt: 73,794 DMemSt: 17,423 Stall: 73,844 (93%) Visible: 91-106 / 41018
```

GEM5 · STATS.TXT, COUNTED DURING THE RUN

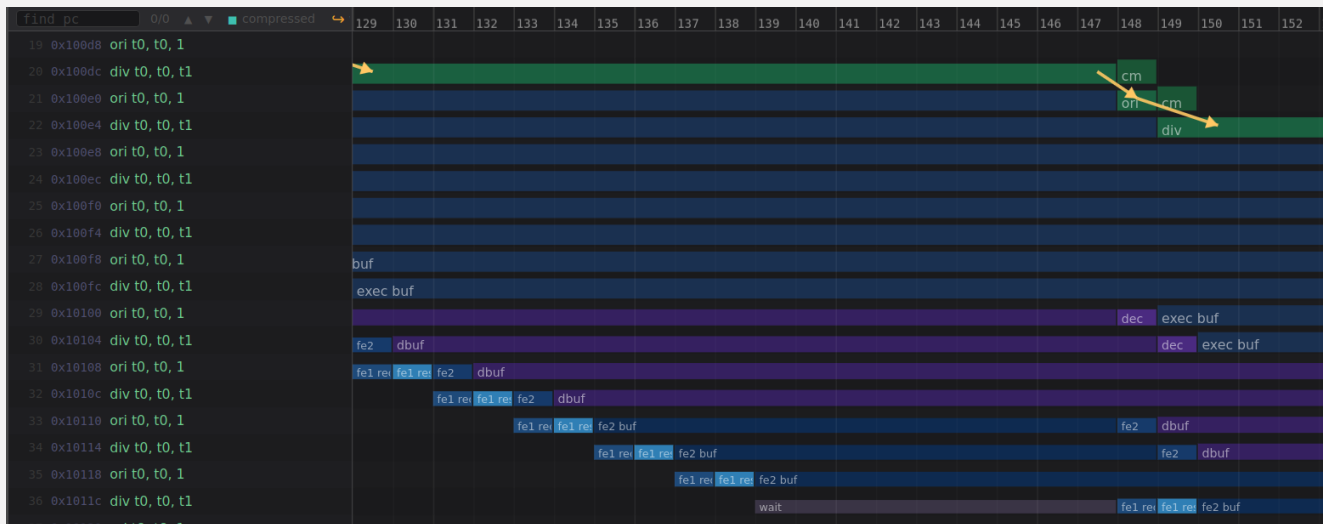
```
board.processor.cores.core.commitStats0.numInsts      40985 # Number of instructions committed (thread level)
board.processor.cores.core.ipc                        0.518719 # IPC: instructions per cycle (core level)
board.cache_hierarchy.l1dcaches.ReadReq.misses::total 1028  # number of ReadReq misses
```

Case A: Functional-unit latency and backpressure

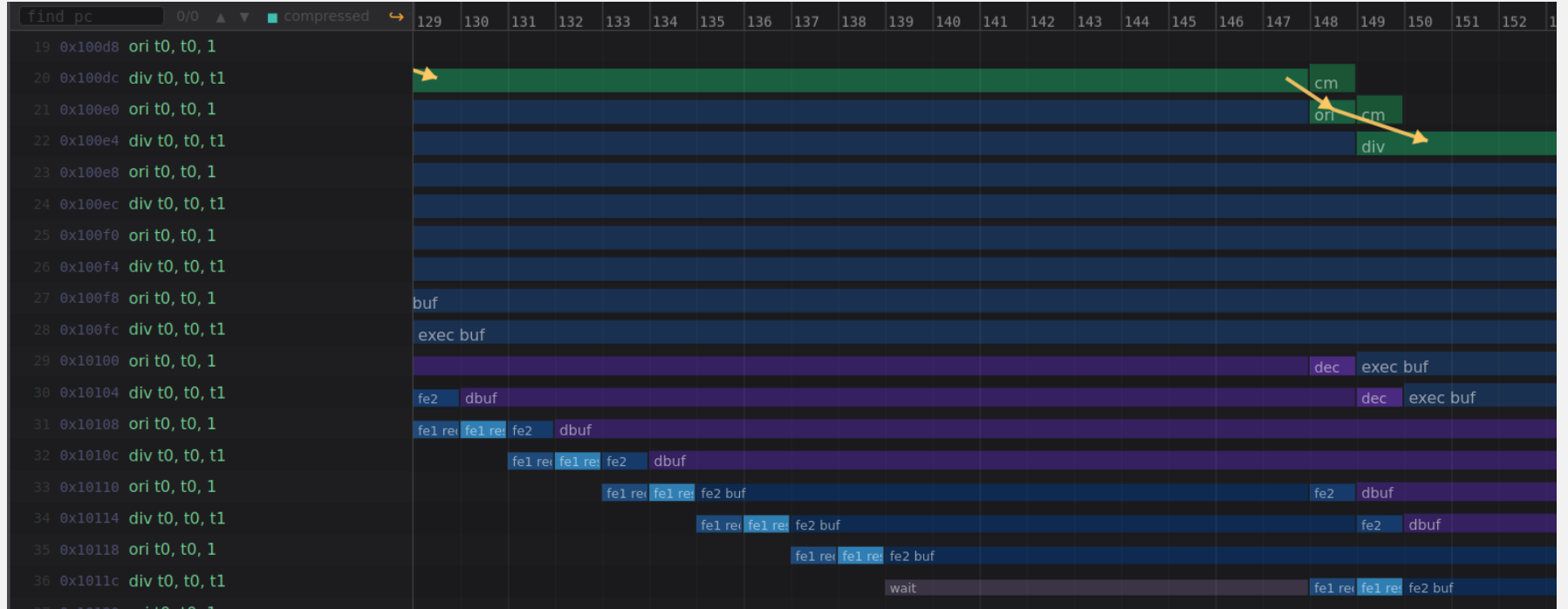
loop:

```
ori t0, t0, 1
div t0, t0, t1
ori t0, t0, 1
div t0, t0, t1
...
```

What to look for: the long green divide bars hold Execute for tens of cycles while the queues behind them saturate at their configured depths of 8, 4 and 3 entries, until the front-end stalls at cycle 139.



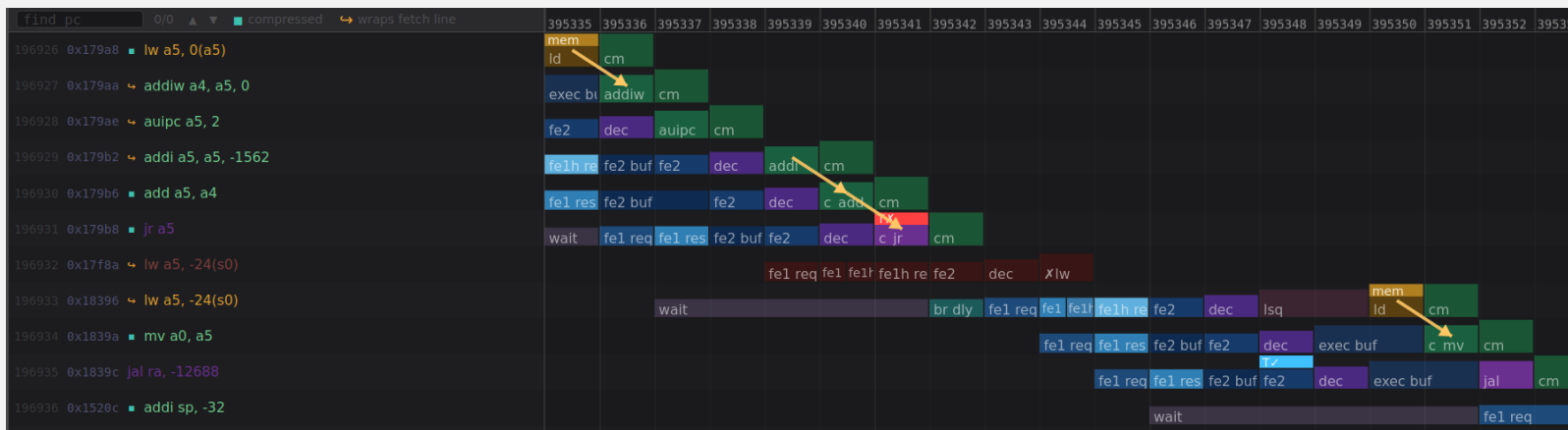
CASE A · DIVIDE BACKPRESSURE · IN DETAIL



Case B: Branch resolution and pipeline flushes

```
0x179b6  add  a5, a4
0x179b8  jr    a5          indirect, mispredicted
0x17f8a  lw    a5, -24(s0)   stale target, flushed
0x18396  lw    a5, -24(s0) the real target
0x1839c  jal   ra, -12688    predicted taken
```

What to look for: `jr` takes its target from a register, but Fetch2 predicts a stale one, `0x17f8a`. The jump resolves at Execute and is flagged mispredicted, that row is killed at cycle 395344, and fetching restarts from the real target after a branch delay. Two rows down, the `jal` carries a predicted-taken mark.



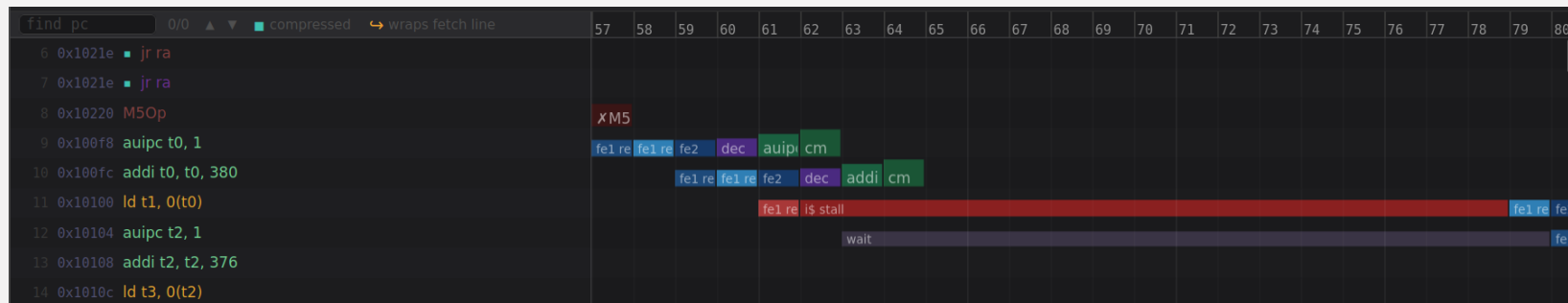


Case C: Instruction-cache miss

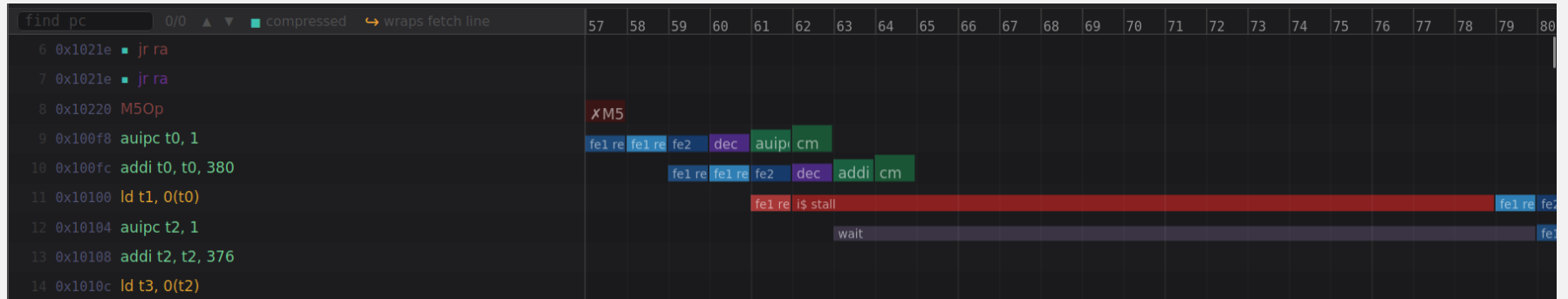
```
0x100f8  auipc t0, 1
0x100fc  addi  t0, t0, 380
0x10100  ld    t1, 0(t0)    fetch misses in the I-cache
0x10104  auipc t2, 1
0x10108  addi  t2, t2, 376
```

What to look for: the red bar running from cycle 61 to 79 is the front-end stalled on the miss.

Eighteen cycles, measurable off the ruler, with every following row held behind it.



CASE C · I-CACHE MISS · IN DETAIL



The Computer Architecture laboratory at FaMAF

The laboratory today

Students run a set of programs under gem5 across several configurations, varying data-cache size and associativity and switching between local and tournament predictors, then collect the matching entries of `stats.txt` and explain why the metrics change.

Integration costs one debug flag and one command. The student-facing workflow does not change.

What the evidence allows

The questions are the right ones and the evidence is a count. A student who observes that a direct-mapped cache underperforms has the miss rate, but not the events that produced it, and the explanation rests on what is plausible rather than on what is visible.

Conclusions and future work

- A streaming Python tracer and a single-file browser viewer turn MinorTrace into a multi-cycle pipeline diagram, with no installation.
- The rendered timeline agrees with gem5's own counters, and the three cases expose behaviour the aggregate statistics leave implicit.
- Next: a controlled study in the FaMAF laboratory, comparing the explanations students produce for the same configuration sweep with and without the tool.

Thank you

MinorFlow is open source. Questions welcome.

github.com/FaMAF-CVA6-Project

MANUEL NIETO

manuel.nieto@mi.unc.edu.ar

MARÍA DELFINA VÉLEZ IBARRA

delfinavelez@unc.edu.ar

FRANCISCO CORTEZ CASINI

fcortez016@mi.unc.edu.ar

GONZALO TOMÁS VODANOVIC

gvodanovic@unc.edu.ar

Presented by Manuel Nieto and Francisco Cortez Casini

